

Universidade do Minho

Escola de Engenharia

Guia 2 – Remote PWM for LED brightness and motor speed

Arquitetura de Computadores II

PL8:

Isabel Gomes – a104868

Joana Carvalho – a104865



1. Descrição do algoritmo

O sistema implementa um controlador de saída PWM comandado por protocolo UART.

1.1. Inicialização

Após *Reset*, o sistema executa a seguinte sequência de inicialização:

1. Configuração do SYSCLK para 49 MHz, desativação do *Watchdog Timer*.
2. Configuração do *Crossbar* (XBAR): ativação de UART0 (P0.4/P0.5) e saída PCA0 CEX0 (P2.0).
3. Configuração dos pinos de I/O: P0.4 e P2.0 como saídas *push-pull*.
4. Inicialização dos *buffers* FIFO circulares de TX e RX (32 bytes cada).
5. Configuração do UART0 a 115200 bps, 8N1, usando Timer1 em modo *auto-reload*.
6. Inicialização do contador PCA0 com fonte SYSCLK/12 (para uma frequência constante) ou pelo *overflow* do timer0 (para uma frequência variável).
7. Inicialização do módulo PWM (Módulo 0 do PCA0, 8 bits).
8. Inicialização do timer de sequência (Módulo 1 do PCA0, *software timer*).
9. Inicialização do *parser* de comandos e do controlador de estado.
10. Ativação global de interrupções (IE_EA = 1).

1.2. Ciclo Principal

O *main loop* executa continuamente as seguintes operações:

- Leitura de um *byte* do *buffer* RX (`uart_getchar`). Se vazio, continua o ciclo.
- Processamento *byte a byte* pela máquina de estados do *parser* (`cmd_process`). Enquanto a trama não estiver completa, continua a aguardar.
- Quando a trama está completa e válida, o *parser* identifica o tipo de comando e o valor associado (`cmd_parser`).
- O controlador executa a ação correspondente e atualiza o estado do sistema (`ctrl_execute`).

1.3. Máquina de Estados do *parser* (`commands.c`)

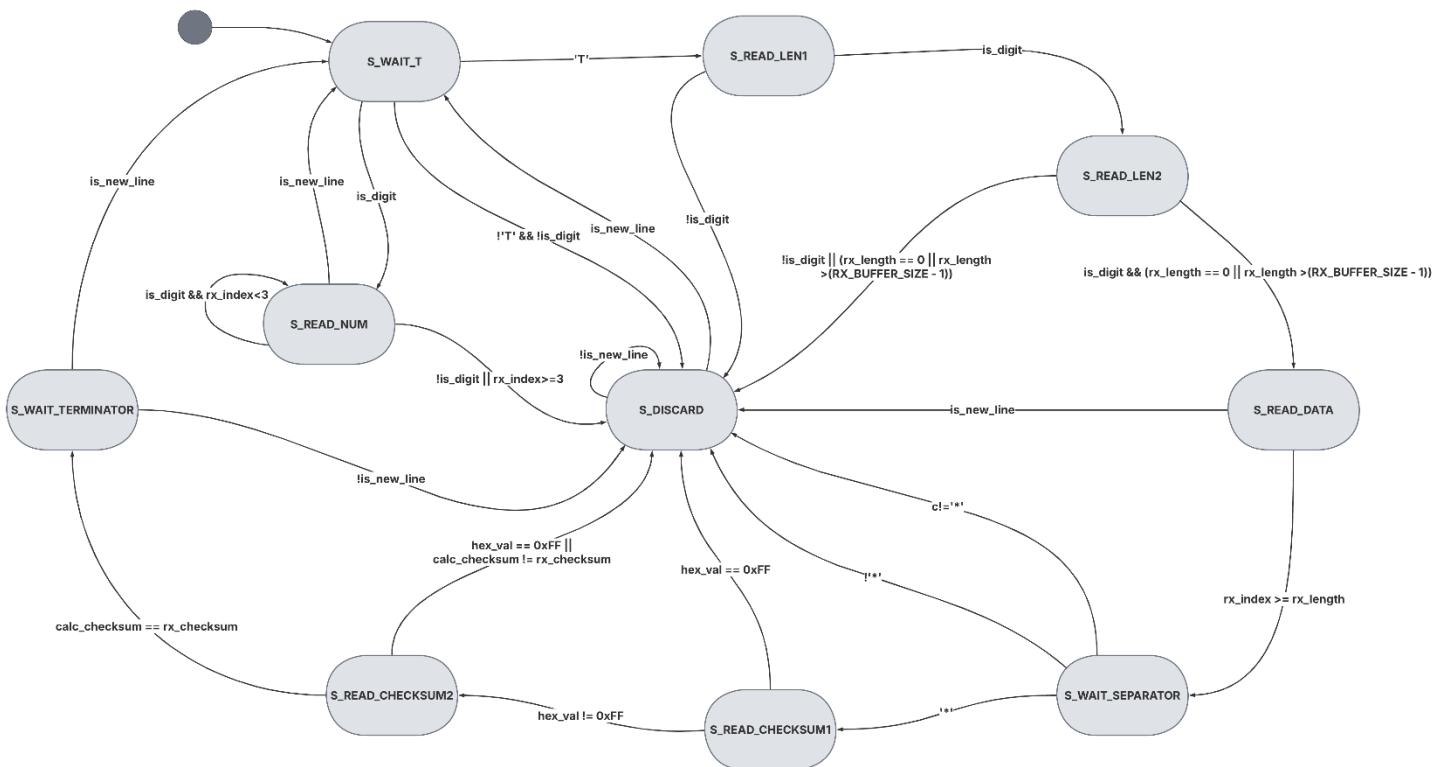
O *parser* processa cada *byte* recebido através de uma máquina de estados com os seguintes estados:

Estado	Descrição
S_WAIT_T	Aguarda o carácter 'T' (início de trama) ou dígito (modo sequência)
S_READ_LEN1	Lê o 1.º dígito decimal do campo de comprimento cc



S_READ_LEN2	Lê o 2.º dígito decimal do campo de comprimento cc
S_READ_DATA	Lê os <dados> <i>byte a byte</i> (rx_length bytes)
S_WAIT_SEPARATOR	Aguarda o carácter '*' (separador de <i>checksum</i> , Fase 2)
S_READ_CHECKSUM1	Lê o 1.º dígito hexadecimal do <i>checksum</i>
S_READ_CHECKSUM2	Lê o 2.º dígito hexadecimal do <i>checksum</i>
S_WAIT_TERMINATOR	Aguarda o terminador '\n'
S_READ_NUM	Lê um valor numérico simples (ponto de sequência)
S_DISCARD	Descarta trama inválida até '\n'; envia ERR uma vez

Máquina de Estados:

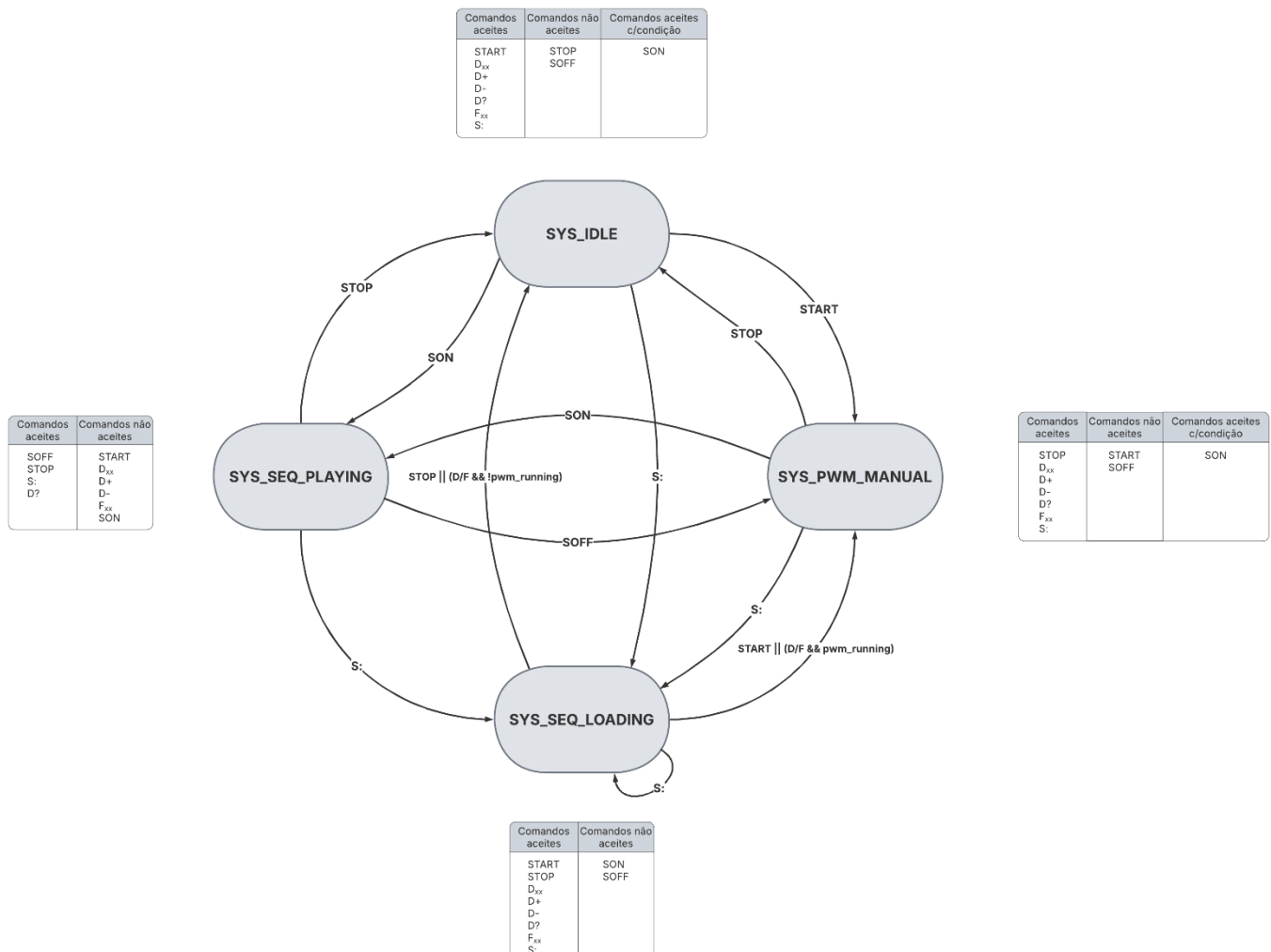


1.4. Máquina de Estados do sistema (system_state.c)

O sistema mantém um estado global que controla quais os comandos permitidos em cada momento:

Estado	Descrição	Transições Principais
SYS_IDLE	PWM parado, sem sequência ativa	START -> PWM_MANUAL; S:xx -> SEQ_LOADING; SON -> SEQ_PLAYING
SYS_PWM_MANUAL	PWM ativo, controlo manual de duty-cycle	STOP -> IDLE; S:xx -> SEQ_LOADING; SON -> SEQ_PLAYING
SYS_SEQ_LOADING	A receber pontos da sequência	Conclusão -> IDLE/PWM_MANUAL; START/STOP/D -> cancela
SYS_SEQ_PLAYING	Sequência em reprodução automática	SOFF -> PWM_MANUAL; STOP -> IDLE; S:xx -> SEQ_LOADING

Máquina de Estados:





1.5. Gestão da sequência (sequence.c)

O módulo de sequência mantém uma máquina de estados interna (SEQ_IDLE, SEQ_LOADING, SEQ_READY, SEQ_PLAYING). O carregamento inicia com o comando S:xx, que define o número de pontos esperados. Os pontos são enviados como valores numéricos ASCII simples, um por linha. Quando todos os pontos são recebidos, a sequência fica no estado SEQ_READY. O comando SON inicia a reprodução: a ISR do PCA0 (Módulo 1) aplica cada ponto automaticamente ao PWM. Opcionalmente, pode ocorrer com um intervalo de aproximadamente 1 segundo, ou a cada interrupção. O comando SOFF para a reprodução da sequência, e volta ao estado SEQ_READY. A sequência continua carregada até introduzir o comando S:xx novamente.

1.6. Fluxo de tratamento de Interrupções

O sistema utiliza duas ISR principais:

- ISR UART0 (vetor 4): Na receção (RI=1), guarda o *byte* diretamente na FIFO de RX sem passar pelo *main loop*. Na transmissão (TI=1), envia o próximo *byte* da FIFO de TX. Garante comunicação sem bloqueio.
- ISR PCA0 (vetor 11): Trata o *compare match* do Módulo 1 (CCF1). Aplica o próximo ponto da sequência com a função `seq_apply_next()`. Limpa a *flag* de *overflow* do contador (CF).

2. Variáveis em memória, Tipos e Periféricos

2.1. Módulo `commands.c` – *Parser* de tramas

Variável	Tipo	Memória	Descrição
<code>rx_data[]</code>	unsigned char[16]	DATA/XDATA	Buffer de <i>payload</i> recebido (<dados>)
<code>rx_length</code>	unsigned char	DATA	Comprimento esperado dos dados (campo <i>cc</i>)
<code>rx_index</code>	unsigned char	DATA	Posição atual de escrita no <i>buffer</i>
<code>rx_state</code>	<code>rx_state_t</code> (enum)	DATA	Estado atual da máquina de receção
<code>rx_checksum</code>	unsigned char	DATA	<i>Checksum</i> recebido na trama (hh)
<code>calc_checksum</code>	unsigned char	DATA	<i>Checksum</i> calculado localmente
<code>len_digit_count</code>	unsigned char	DATA	Contador de dígitos lidos no campo <i>cc</i>
<code>checksum_digit_count</code>	unsigned char	DATA	Contador de dígitos lidos no <i>checksum</i>
<code>error_sent</code> (static)	bit	BIT	Flag para enviar ERR apenas uma vez por trama

2.2. Módulo `PWM.c` – Controlo do *Duty-Cycle*

Variável	Tipo	Memória	Descrição
<code>current_duty</code>	unsigned char	DATA	<i>Duty-cycle</i> atual em percentagem (0–100%)
<code>pwm_running</code>	volatile bit	BIT	Flag: 1 se PWM ativo, 0 se parado



2.3. Módulo `sequence.c` – Gestão da Sequência

Variável	Tipo	Memória	Descrição
<code>sequence_buffer[]</code>	<code>unsigned char[30]</code>	DATA/XDATA	Array com os 30 valores de <i>duty-cycle</i>
<code>seq_state</code>	<code>seq_state_t</code> (enum)	DATA	Estado da máquina de sequência
<code>seq_total_points</code>	<code>unsigned char</code>	DATA	Número total de pontos a receber
<code>seq_loaded_points</code>	<code>unsigned char</code>	DATA	Número de pontos já carregados
<code>seq_playback_index</code>	<code>unsigned char</code>	DATA	Índice do próximo ponto a aplicar

2.4. Módulo `driverPCA0.c` – *Timer* de Sequência

Variável	Tipo	Memória	Descrição
<code>pwm_cycle_count</code>	<code>volatile unsigned int</code>	DATA	Contador de ciclos do Módulo 1 para temporização de 1 segundo



2.5. Módulo driverUART.c – *Buffers* de Comunicação

Variável	Tipo	Memória	Descrição
TxBuff[]	unsigned char[32] pdata	XDATA (pdata)	<i>Buffer</i> físico da FIFO de transmissão
RxBuff[]	unsigned char[32] pdata	XDATA (pdata)	<i>Buffer</i> físico da FIFO de receção
tx_fifo	fifo_t	DATA	Estrutura de controlo da FIFO TX
rx_fifo	fifo_t	DATA	Estrutura de controlo da FIFO RX
ti_restart	volatile bit	BIT	<i>Flag</i> para reiniciar transmissão quando FIFO TX fica vazia

2.6. Módulo system_state.c – Estado Global

Variável	Tipo	Memória	Descrição
current_state	system_state_t (enum)	DATA	Estado atual do sistema (IDLE, PWM_MANUAL, SEQ_LOADING, SEQ_PLAYING)



3. Configuração dos Periféricos

3.1. Oscilador e SYSCLK(init.c)

O sistema opera a 49 MHz usando o oscilador interno HFOSC1. A configuração é efetuada da seguinte forma:

- *Watchdog Timer* desativado: WDTCN = 0xDE seguido de WDTCN = 0xAD.
- Mudança para SFRPAGE = 0x10 para acesso ao registo PFE0CN.
- PFE0CN |= FLRT__SYSCLK_BELOW_50_MHZ — configura a memória *Flash* para SYSCLK < 50 MHz.
- CLKSEL |= CLKSL__HFOSC1 — seleciona HFOSC1 como fonte de relógio do sistema.

3.2. Crossbar e I/O (init.c)

O Crossbar roteia os periféricos internos para os pinos físicos:

Registo	Valor	Função
XBR2	XBARE__ENABLED	Ativa o <i>Crossbar</i>
XBR0	URT0E__ENABLED	Roteia UART0 para P0.4 (TX) e P0.5 (RX)
XBR1	PCA0ME__CEX0	Roteia PCA0 CEX0 para o próximo pino disponível (P2.0)
P0MDOUT	bit 4 = 1	P0.4 (TX) como saída <i>push-pull</i>
P2MDOUT	bit 0 = 1	P2.0 (CEX0/PWM) como saída <i>push-pull</i>
P0SKIP	0xCF	Salta P0.0-P0.3 e P0.6-P0.7; UART em P0.4-P0.5
P1SKIP	0xFF	Salta todo o <i>Port 1</i>

3.3. UART0 (driverUART.c)

Configurado a 115200 bps, formato 8N1, usando *Timer1* como gerador de *baudrate*:

Registo	Valor	Função
CKCON0	T1M__SYSCLK	<i>Timer1</i> usa SYSCLK como fonte (49 MHz)
TMOD	T1M__MODE2	<i>Timer1</i> em modo 2 (<i>auto-reload</i> de 8 bits)
TH1 / TL1	43	Valor de <i>reload</i> para 115200 bps com SYSCLK=49MHz
SCON0_REN	1	Ativa a receção UART0
SCON0_TI	1	Ativa a transmissão UART0
IE_ES0	1	Ativa interrupção da UART0

A comunicação é totalmente por interrupção. A ISR (vetor 4) gere separadamente os *flags* RI (receção) e TI (transmissão), usando as FIFOs circulares como interface entre a ISR e o código de aplicação.

3.4. PCA0 – Contador Base (driverPC0.C)

Registo	Valor	Função
PCA0CN0_CR	0 -> 1	Para o contador, configura, depois inicia
PCA0CN0_CF	0	Limpa <i>flag</i> de <i>overflow</i>
PCA0MD	ECF__OVF_INT_ENABLED	Ativa interrupção de <i>overflow</i> do contador
PCA0MD	CPS__SYSCLK_DIV_12	Fonte de relógio: SYSCLK/12 (~4.08 MHz)
EIE1	EPCA0__ENABLED	Ativa interrupção global do PCA0

3.5. PCA0 – Módulo 0: Saída PWM 8 bits (PWM.c)

O Módulo 0 gera a saída PWM no pino CEX0 (P2.0):

Registo	Configuração ativa	Função
PCA0CPM0	ECOM__ENABLED PWM__ENABLED	Ativa comparador e modo PWM de 8 bits
PCA0CPH0	duty_percent * 255 / 100	Registo de comparação: define o <i>duty-cycle</i>
PCA0CPL0	0x7F (inicia a 50%)	Parte baixa do registo de comparação

Para parar o PWM, PCA0CPM0 = 0 desativa o comparador sem parar o contador PCA0.

3.6. PCA0 – Módulo 1: *Software Timer* de Sequência (driverPCA0.c)

O Módulo 1 funciona como *software timer* para cadenciar os pontos da sequência:

Registo	Valor	Função
PCA0CPM1	ECOM MAT ECCF	Ativa comparador, <i>compare match</i> e interrupção CCF1
PCA0CPL1	0x00	Parte baixa do registo de comparação inicial
PCA0CPH1	0x01	Parte alta do registo de comparação inicial

3.7. Estrutura FIFO (fifo.c)

A FIFO é implementada com índices de leitura e escrita circulares (obtidos pela máscara ($size - 1$), o que exige que o tamanho seja uma potência de 2). O número de elementos é dado por ($index_write - index_read$). As funções *fifo_push_data* e *fifo_pop_data* são protegidas contra interrupções com a diretiva *#pragma disable*.



4. Programa Final em Linguagem C

O projeto está organizado nos seguintes módulos:

Ficheiro	Responsabilidade
main.c	Inicialização do sistema e ciclo principal
init.c/h	Configuração de <i>hardware</i> : SYSCLK, Crossbar, I/O
driverUART.c/h	Driver UART0 com FIFOs circulares e ISR
fifo.c/h	Estrutura FIFO genérica com índices circulares
driverPCA0.h	Interface pública do PCA0 (declarações)
driverPCA0.c	Inicialização do PCA0 e do módulo de Software timer e ISR
PWM.c	Módulo 0 do PCA0: geração e controlo do sinal PWM
sequence.c	Módulo 1 do PCA0: carregamento e reprodução de sequência
commands.c/h	<i>Parser</i> de tramas ASCII: máquina de estados <i>byte a byte</i>
controller.c/h	Lógica de execução de comandos com validação de estado
system_state.c/h	Máquina de estados global do sistema (4 estados)

5. Fase 2 – Extensão com Checksum

5.1. Estrutura da Trama na Fase 2

Na Fase 2, o protocolo de comunicação foi estendido com um campo de verificação de integridade. A trama passou a ter o seguinte formato:

Tcc <dados>*hh\n

Campo	Exemplo	Descrição
T	T	Carácter de início de trama (fixo)
cc	03	Comprimento dos <dados> em 2 dígitos decimais ASCII
<dados>	D75	Conteúdo útil da trama (comando/parâmetro)
*	*	Separador entre dados e checksum
hh	B0	<i>Checksum</i> em 2 dígitos hexadecimais ASCII (maiúsculas)
\n	\n	Terminador de trama

Exemplo completo: T03D75*B0\n

- Dados: 'D', '7', '5' → $0x44 + 0x37 + 0x35 = 0xB0$
- Checksum* enviado: B0 — coincide com o calculado, trama aceite.

5.2. Algoritmo de Cálculo do *Checksum*

O *checksum* é calculado como a soma aritmética dos valores ASCII de todos os *bytes* dos <dados>. Apenas aceita dois caracteres, logo, o resultado é de apenas 8 bits (menos significativos).

```
static unsigned char calculate_checksum(void)
{
    unsigned char sum = 0;
    unsigned char i;
    for (i = 0; i < rx_length; i++)
        sum += rx_data[i];      // soma valores ASCII de cada byte
    return sum;                 // resultado de 8 bits
}
```

Esta abordagem é simples: deteta erros de *bytes* alterados individualmente, embora não detete todas as combinações de erros múltiplos.

5.3. Verificação na Máquina de Estados

A Fase 2 introduz dois estados adicionais na máquina de receção: `S_READ_CHECKSUM1` e `S_READ_CHECKSUM2`. Após receber os <dados> completos, o estado `S_WAIT_SEPARATOR` aguarda o carácter '*'. De seguida, os dois dígitos hexadecimais do *checksum* são lidos e combinados:

```
case S_READ_CHECKSUM1:
    hex_val = hex_to_value(c);
    rx_checksum = hex_val << 4;           // 4 bits mais significativos
    rx_state = S_READ_CHECKSUM2;
    break;

case S_READ_CHECKSUM2:
    hex_val = hex_to_value(c);
    rx_checksum |= hex_val;              // 4 bits menos significativos
    calc_checksum = calculate_checksum();
    if (calc_checksum != rx_checksum) { rx_state = S_DISCARD; break; }
    rx_state = S_WAIT_TERMINATOR;       // checksum valido
    break;
```

A implementação da Fase 2 é totalmente integrada na mesma máquina de estados da Fase 1. O *parser* trata os novos estados de *checksum* de forma transparente: se o carácter '*' não aparecer após os dados (trama sem *checksum*), o *parser* transita para `S_DISCARD` e envia ERR. Desta forma, apenas tramas com *checksum* válido são aceites.

5.4. Tabela de Comandos com Checksum (cmd_notes.txt)

Para facilitar o teste do protocolo, a tabela seguinte lista os comandos implementados com os respetivos *checksums* pré-calculados:

Trama completa	Dados	Checksum(hex)	Descrição
T02D0*74\n	D0	0x74	Define <i>duty-cycle</i> para 0%
T03D50*A9\n	D50	0xA9	Define <i>duty-cycle</i> para 50%
T03D75*B0\n	D75	0xB0	Define <i>duty-cycle</i> para 75%
T04D100*D5\n	D100	0xD5	Define <i>duty-cycle</i> para 100%
T02D+*6F\n	D+	0x6F	Incrementa <i>duty-cycle</i> em 1%
T02D-*71\n	D-	0x71	Decrementa <i>duty-cycle</i> em 1%
T02D?*83\n	D?	0x83	Consulta estado e <i>duty-cycle</i> atual
T05START*8E\n	START	0x8E	Ativa saída PWM
T04STOP*46\n	STOP	0x46	Desativa saída PWM
T04F100*D7\n	F100	0xD7	Define frequência para 100 Hz
T04F500*DB\n	F500	0xDB	Define frequência para 500 Hz
T05F1000*07\n	F1000	0x07	Define frequência para 1000 Hz
T03S:1*BE\n	S:1	0xBE	Inicia carregamento de 1 ponto
T03S:5*C2\n	S:5	0xC2	Inicia carregamento de 5 pontos
T04S:10*EE\n	S:10	0xEE	Inicia carregamento de 10 pontos
T03SON*F0\n	SON	0xF0	Inicia reprodução da sequência
T04SOFF*2E\n	SOFF	0x2E	Para reprodução da sequência